

Title

PL/SQL Procedures and Functions

Objectives

1. To understand PL/SQL stored procedures and functions.
 2. To implement a stored procedure to insert sales records.
 3. To create a function to calculate total sales.
 4. To improve data handling efficiency using database-side programming.
 5. To demonstrate modular and reusable PL/SQL program units.
-

Problem Statement

Design and implement a **Sales Management System** in PL/SQL that:

- Stores sales transaction details in a database table.
 - Uses a **stored procedure** to insert sales records.
 - Uses a **function** to calculate and return total sales.
 - Displays output tables showing inserted and calculated results.
-

Outcomes

After completion of this experiment, students will be able to:

- Create and execute stored procedures.
 - Create and execute PL/SQL functions.
 - Handle data efficiently within the database.
 - Perform aggregate calculations using PL/SQL.
 - Improve modular programming in Oracle SQL.
-

Software and Hardware Requirements

Software

- Oracle Database (e.g., Oracle Database)
- SQL*Plus / Oracle SQL Developer

Hardware

- Minimum 4GB RAM
- Intel i3 or higher processor
- 20GB free disk space

Theory Concept (Brief)

◆ PL/SQL

PL/SQL (Procedural Language/SQL) is Oracle's procedural extension of SQL that allows developers to write structured programs including loops, conditions, procedures, and functions.

◆ Stored Procedure

A stored procedure is a named PL/SQL block stored in the database and executed when called. It performs specific tasks such as inserting, updating, or deleting records.

◆ Function

A function is a PL/SQL block that returns a value. It is mainly used for calculations and can be used inside SQL statements.

Database Design

Table: SALES

Column Name	Data Type	Description
SALE_ID	NUMBER	Primary Key
PRODUCT_NAME	VARCHAR2(50)	Name of product
QUANTITY	NUMBER	Quantity sold
PRICE	NUMBER(10,2)	Price per unit
SALE_DATE	DATE	Date of sale

Implementation

1 Create Table

```
CREATE TABLE SALES (  
    SALE_ID NUMBER PRIMARY KEY,  
    PRODUCT_NAME VARCHAR2(50),  
    QUANTITY NUMBER,  
    PRICE NUMBER(10,2),  
    SALE_DATE DATE  
);
```

2 Stored Procedure to Add Sales Data

```

CREATE OR REPLACE PROCEDURE ADD_SALE(
    P_ID NUMBER,
    P_NAME VARCHAR2,
    P_QTY NUMBER,
    P_PRICE NUMBER,
    P_DATE DATE
)
IS
BEGIN
    INSERT INTO SALES
    VALUES (P_ID, P_NAME, P_QTY, P_PRICE, P_DATE);

    COMMIT;
END;
/

```

3 Function to Calculate Total Sales

```

CREATE OR REPLACE FUNCTION GET_TOTAL_SALES
RETURN NUMBER
IS
    TOTAL_SALES NUMBER;
BEGIN
    SELECT SUM(QUANTITY * PRICE)
    INTO TOTAL_SALES
    FROM SALES;

    RETURN TOTAL_SALES;
END;
/

```

Algorithm

◆ Procedure to Add Sales

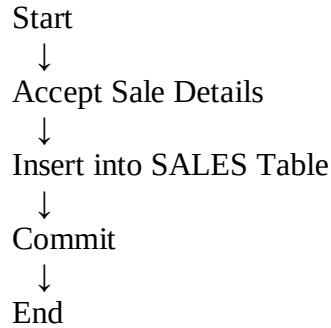
1. Start
2. Accept sale details as input
3. Insert values into SALES table
4. Commit transaction
5. End

◆ Function to Calculate Total Sales

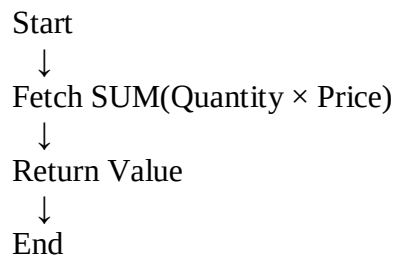
1. Start
2. Retrieve $SUM(QUANTITY \times PRICE)$ from SALES
3. Store result in variable
4. Return total value
5. End

Flowchart

Add Sale Procedure



Total Sales Function



Test Data Set

```
BEGIN
  ADD_SALE(1, 'Laptop', 2, 50000, SYSDATE);
  ADD_SALE(2, 'Mouse', 5, 500, SYSDATE);
  ADD_SALE(3, 'Keyboard', 3, 1000, SYSDATE);
END;
/
```

Output Tables

SALES Table After Insertion

SALE_ID	PRODUCT_NAME	QUANTITY	PRICE	SALE_DATE
1	Laptop	2	50000	02-MAR-2026
2	Mouse	5	500	02-MAR-2026
3	Keyboard	3	1000	02-MAR-2026

Total Sales Output

```
SELECT GET_TOTAL_SALES FROM DUAL;
```

Output:

GET_TOTAL_SALES

105500

Calculation:

- Laptop $\rightarrow 2 \times 50000 = 100000$
- Mouse $\rightarrow 5 \times 500 = 2500$
- Keyboard $\rightarrow 3 \times 1000 = 3000$

Total = 105500

Test Cases

Test Case ID	Input	Expected Result	Status
TC1	Valid sale data	Record inserted	Pass
TC2	Calculate total after insertion	Correct sum displayed	Pass
TC3	Insert duplicate SALE_ID	Error (Primary Key violation)	Pass
TC4	No records in table	Function returns NULL	Pass

Conclusion

The Sales Management System was successfully designed and implemented using PL/SQL stored procedures and functions. The stored procedure efficiently inserts sales data, and the function calculates total sales dynamically. This implementation demonstrates how PL/SQL improves modularity, efficiency, and centralized data management within Oracle databases.